Universidade da Beira Interior

Departamento de Informática

Jogo da Forca Multijogador

Elaborado por:
Alexandre Santos (52011)
Daniel Carlos (52578)
Vasco Colaço (52290)

Unidade Curricular:
Sistemas Distribuídos

14 de abril de 2026

Conteúdo

1 Introdução **5**
1.1 Enquadramento 5
1.2 Objetivos 5
1.3 Organização do Documento 5

2 Tecnologias e Ferramentas Utilizadas **7**
2.1 Introdução 7
2.2 Paralelismo Assíncrono 7
2.3 A Garantia TCP/IP 7

3 Implementação **8**
3.1 Introdução 8
3.2 Estruturação Arbórea do Código de Fonte 8
3.3 Core Lógico (GameEngine e Sincronismo) 8
3.4 O Protocolo Lexical 9
3.4.1 Mensagens Servidor → Cliente 9
3.4.2 Mensagens Cliente → Servidor 10
3.5 Sincronização e Exclusão Mútua 10
3.6 Diagrama de Sequência 10
3.7 Tratamento de Erros e Tolerância a Falhas 12
3.8 Pequenas Funcionalidades 12
3.8.1 Contagem Regressiva Dinâmica em Consola (Códigos ANSI) 12
3.8.2 Renderização Gráfica do Boneco 13
3.8.3 Apresentação de Letras Tentadas Anteriormente 14

4 Testes e Validação do Sistema **15**
4.1 Gestão de Lotação (Servidor Cheio) 15
4.2 Resiliência a Falhas de Rede (Desconexão) 15

5 Conclusões e Trabalho Futuro **16**
5.1 Conclusão 16
5.2 Trabalho Futuro 16

Lista de Figuras

3.1 Ciclo de Mensagens Típico entre um Cliente e o Servidor 11

4.1 Interface do Cliente ao tentar entrar num lobby já completo ou em jogo. . . . 15
4.2 Notificação visual de desconexão inesperada de um dos participantes. 15

Acrónimos

TCP Transmission Control Protocol

IP Internet Protocol

UBI Universidade da Beira Interior

UDP User Datagram Protocol

Glossário

Socket Ponto final de uma ligação de comunicação bidirecional entre dois programas através de uma rede de dados.

Thread Linha de execução independente e secundária dentro de um processo principal, permitindo a concorrência computacional.

Capítulo 1

Introdução

1.1 Enquadramento

No âmbito prático de programação e implementação de sistemas de rede, foi proposta a construção de um sistema distribuído baseado na típica arquitetura cliente-servidor. O projeto selecionado consiste no desenvolvimento do **Jogo da Forca Multijogador**, utilizando a linguagem de programação Java e a comunicação de dados bidirecional assente em *sockets* TCP.

Este tipo de arquitetura é utilizada em aplicações da atualidade, que requerem comunicação extremamente fiável e trocas de dados em tempo real.

1.2 Objetivos

O objetivo principal desta proposta é criar um servidor resistente que coordene várias partidas do Jogo da Forca ao mesmo tempo, permitindo que vários jogadores participem em simultâneo. Também pretende organizar melhor a comunicação com os clientes e tornar a interface do utilizador mais clara e interativa.

Estes são os principais objetivos do programa:

1. Criar a base do sistema Cliente-Servidor usando *sockets* (ServerSocket e Socket em Java).

2. Permitir várias ligações ao mesmo tempo (entre 2 a 4 jogadores), com início automático do jogo após algum tempo de espera no lobby.

3. Definir um protocolo de comunicação simples, baseado em mensagens de texto trocadas entre cliente e servidor.

4. Garantir que as rondas decorrem corretamente, com tempo limite para cada jogada (*timeout*), e atualizar o estado do jogo (palavra escolhida aleatoriamente de uma lista de 100).

5. Evitar erros e conflitos no jogo, assegurando que a gestão das vidas e das jogadas é feita de forma segura e organizada.

1.3 Organização do Documento

Para descrever melhor os procedimentos, este documento está organizado nas seguintes secções:

- **Capítulo 1 – Introdução** – apresenta de forma geral o tema e os objetivos do programa.
- **Capítulo 2 – Tecnologias e Ferramentas Utilizadas** – descreve as necessidades do projeto e as ferramentas usadas para o desenvolver.
- **Capítulo 3 – Implementação e Testes** – explica como o servidor foi desenvolvido e como funciona o jogo em modo multijogador.
- **Capítulo 4 – Conclusões e Trabalho Futuro** – resume o trabalho realizado e apresenta possíveis melhorias para o futuro.

Capítulo 2

Tecnologias e Ferramentas Utilizadas

2.1 Introdução

O desenvolvimento deste projeto multijogador distribuído, desenvolvido em **Java**, baseou-se na escolha de um conjunto adequado de ferramentas que garantissem o funcionamento consistente entre diferentes ambientes. De seguida, são apresentados os principais elementos técnicos utilizados.

2.2 Paralelismo Assíncrono

Para lidar com vários jogadores ao mesmo tempo e evitar bloqueios durante a comunicação, foram utilizadas *threads*. Cada ligação é tratada de forma independente, permitindo que o servidor continue a funcionar e a aceitar novas conexões sem interrupções.

2.3 A Garantia TCP/IP

De forma a garantir que as mensagens entre jogadores e servidor chegam corretamente e na ordem certa, foi utilizado o protocolo TCP. Este assegura uma comunicação fiável durante o jogo.

Capítulo 3

Implementação

3.1 Introdução

Esta secção descreve as principais decisões de arquitetura e explica como o sistema foi desenvolvido para permitir o funcionamento do jogo em rede.

3.2 Estruturação Arbórea do Código de Fonte

Para organizar melhor o código e manter uma boa estrutura, o projeto foi dividido em três partes principais, cada uma com a sua função:

pt.ubi.sd.forca.shared **(Repositório Comum)** Contém elementos partilhados entre cliente e servidor, como constantes, definições do protocolo e estruturas necessárias para a comunicação.

pt.ubi.sd.forca.server **(Servidor)** Responsável por gerir as ligações dos jogadores, coordenar o jogo e controlar a lógica principal, como palavras, pontuação e regras das rondas.

pt.ubi.sd.forca.client **(Cliente)** Responsável pela interação com o utilizador, mostrando informação no ecrã e enviando comandos para o servidor.

3.3 Core Lógico (GameEngine e Sincronismo)

A lógica principal do jogo é controlada por um componente chamado *GameManager*, criado pelo servidor, que escolhe aleatoriamente uma palavra a partir de uma lista pré-definida com 100 opções. A sessão começa por aceitar ligações de jogadores, iniciando o jogo quando há pelo menos 2 participantes após um *timeout* no lobby, ou imediatamente quando se atinge o máximo de 4 jogadores. Depois do jogo começar, novos jogadores são recusados através da mensagem `FULL` e obrigados a desligar.

Durante o jogo, gerido pelo *GameEngine*, a jogabilidade decorre em rondas síncronas. O estado do jogo é partilhado com todos os participantes. Em cada ronda, é iniciado um cronómetro para que os jogadores submetam uma letra única ou tentem adivinhar a palavra, sendo as respostas avaliadas sem distinção entre maiúsculas e minúsculas. Respostas fora do tempo limite são penalizadas, sendo consideradas como uma submissão de texto vazio ("").

Após todas as respostas serem recebidas, o sistema verifica os acertos, reduz o número de tentativas falhadas até um máximo definido (por exemplo, 6), e determina o fim do jogo quando a palavra é descoberta ou quando se esgotam as tentativas.

Para ilustrar o processamento de uma tentativa, o seguinte bloco de código demonstra como o motor de jogo (*GameEngine*) analisa tanto caracteres individuais como tentativas completas de descobrir a palavra, reduzindo as vidas em caso de falha:

```java
public boolean processGuess(String guess) {
    guess = guess.toUpperCase();
    if (guess.length() == 1) {
        char letter = guess.charAt(0);
        usedLetters.add(letter);
        if (targetWord.indexOf(letter) >= 0) return true;
    } else if (guess.equals(targetWord)) {
        // Acertou a palavra toda
        for (char c : targetWord.toCharArray()) usedLetters.add(c);
        return true;
    }
    attempts--;
    return false;
}
```

Listing 3.1: Processamento de palpites na classe GameEngine

3.4 O Protocolo Lexical

A comunicação entre Servidor e Cliente baseia-se em mensagens de texto, onde cada comando ocupa uma linha completa terminada por um *line break*.

3.4.1 Mensagens Servidor → Cliente

O Servidor envia comandos essenciais para controlar o jogo e informar os Clientes:

- `WELCOME <id> <total>` – Cumprimento inicial, atribuindo a posição de cada jogador na sessão.
- `LOBBY_UPDATE <total> <time>` – Notifica os jogadores no lobby do estado da sala (número de conexões) e decorrer do temporizador até iniciar o jogo.
- `START <mask> <attempts> <timeout>` – Indica o início do jogo, despachando as regras da partida (como as tentativas máximas calculadas em grupo).
- `ROUND <k> <mask> <attempts> <used>` – Informa sobre o novo ciclo e o estado atual.
- `STATE <mask> <attempts> <used>` – Atualização do estado do jogo para sincronização.
- `END WIN <winners> <word>` – Finaliza a partida indicando os vencedores e a palavra correta.
- `END LOSE <word>` – Finaliza a partida informando a palavra correta quando todos falharam.
- `FULL` – Rejeita jogadores que tentam entrar após a partida começar.
- `CANCELLED` – Notifica que o lobby foi cancelado por falta de jogadores.
- `PLAYER_LEFT <id>` – Indica que um jogador se desconectou durante a partida.

3.4.2 Mensagens Cliente → Servidor

O Cliente envia apenas requisições ativas, como:

- `GUESS <text>` – Submissão de uma letra ou da palavra completa.

3.5 Sincronização e Exclusão Mútua

Para garantir a integridade dos dados, foi utilizado o mecanismo de *monitor* do Java através da palavra-chave `synchronized(this)`. Isto permitiu definir secções críticas no *GameManager*, assegurando que, por exemplo, dois jogadores não conseguem ser adicionados à mesma posição da lista simultaneamente ou que o processamento do fim da ronda só ocorre após a receção de todas as jogadas.

O trecho de código abaixo demonstra a aplicação do `synchronized(this)` na adição de jogadores, na classe *GameManager*, avaliando e atualizando a capacidade de rede de forma totalmente atómica face a pedidos paralelos:

```
public void addPlayer(ClientHandler player) {
    synchronized(this) {
        if (players.size() >= 4 || gameStarted) {
            player.sendMessage(Protocol.FULL);
            player.closeConnection();
            return;
        }

        players.add(player);
        int id = players.size() - 1;
        player.setPlayerId(id);

        player.sendMessage(Protocol.WELCOME + " " + id + " " + players.size()
            );
        checkLobbyStatus();
    }
}
```

Listing 3.2: Exclusão Mútua ao adicionar jogadores

3.6 Diagrama de Sequência

O diagrama seguinte representa o fluxo principal de mensagens do protocolo, demonstrando o ciclo de vida de uma partida, desde a ligação inicial de um cliente até ao desfecho final do jogo.

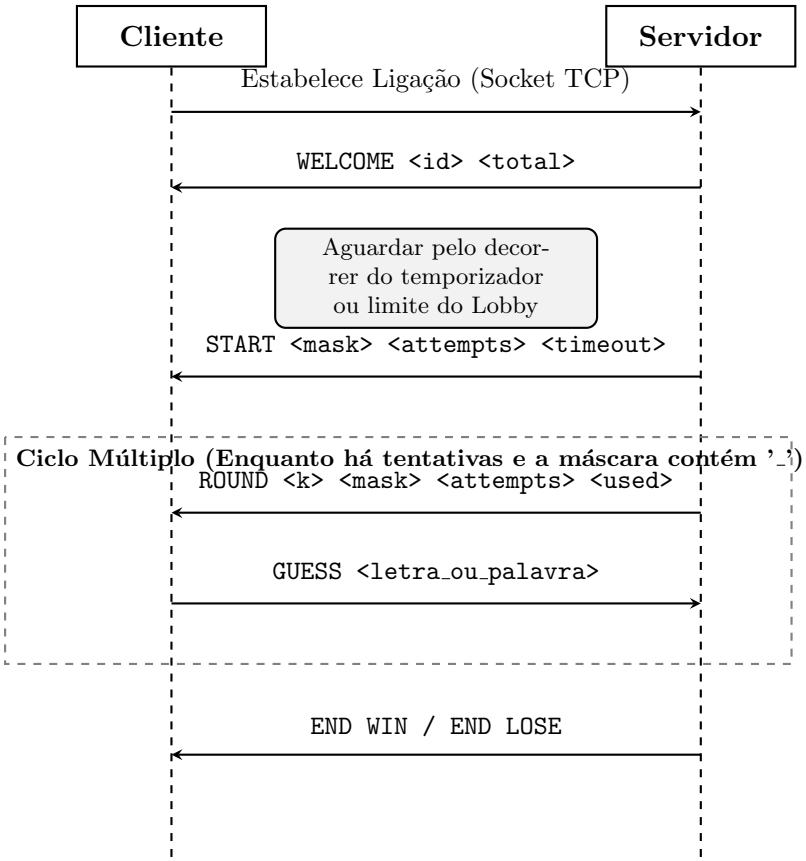

Figura 3.1: Ciclo de Mensagens Típico entre um Cliente e o Servidor

3.7 Tratamento de Erros e Tolerância a Falhas

O servidor foi desenhado para ser capaz de resistir a desconexões inesperadas. Caso um *socket* seja fechado abruptamente, a *thread* dedicada a esse cliente deteta a *IOException*, encerra os recursos localmente e o *GameManager* é notificado para remover o jogador da sessão ativa ou declarar a sua desistência.

Este comportamento foi implementado na função `playerDisconnected`, acionada de forma assíncrona assim que falhas de rede são apanhadas e processadas:

```
public synchronized void playerDisconnected(ClientHandler player) {
    players.remove(player);
    if (gameStarted) {
        // Avisa os restantes que um jogador saiu e termina o jogo
        broadcast(Protocol.PLAYER_LEFT + " " + player.getPlayerId());
        stopGame();
    } else if (players.isEmpty()) {
        // Cancela o timer do lobby se n o houver mais jogadores para
            garantir poupan a do CPU
        if (lobbyTimer != null) {
            lobbyTimer.cancel();
            lobbyTimer = null;
        }
    }
}
```

Listing 3.3: Tratamento de desconexões do servidor para com os jogadores

3.8 Pequenas Funcionalidades

3.8.1 Contagem Regressiva Dinâmica em Consola (Códigos ANSI)

Uma das limitações técnicas inerentes às aplicações de consola que bloqueiam a *thread* principal a aguardar a leitura do teclado (`scanner.nextLine()`) é a incapacidade de lidar com atualizações fluidas na interface. A tentativa de mostrar um relógio tradicional com contagem visível a descer obrigaria a que as letras ativamente digitadas pelo jogador fossem intercetadas ou engolidas.

Para solucionar de forma elegante este desafio visual imposto, aplicámos uma abordagem assente em formatações de **ANSI escape sequences**.

A lógica opera lançando uma *Thread daemon* sempre que se exige uma jogada. A cada decréscimo intervalar atómico de um segundo, utilizam-se estas tags de controlo para manipular a janela (matriz Y) da seguinte forma:

1. `\033[s` guarda na memória da consola a posição exata das coordenadas em que o curso digitável se encontra suspenso, sem imprimir nada de novo.

2. `\033[<N>A` movimenta temporariamente e verticalmente o indicador de ecrã para cima num espaço rígido correspondente a **N** linhas, ou seja, retrocedendo as linhas que desenharam o "Boneco"e restantes textos.

3. Chegando ao ponto calculado pelo algoritmo, sobrepõe toda a linha para uma nova que contém o cronómetro subtraído. Dispara adicionalmente um comando de limpeza limiar (`\033[K`).

4. Imediatamente, e sem que a operação seja percetível à cadência de imagem dos píxeis normais, a tag de fecho `\033[u` força a que o cursor seja reposicionado no fundo para que possa continuar ativamente a processar as teclas em ciclo de *buffer* do *Scanner*.

Na porção técnica da classe de gráfica (`ConsoleUI.java`), todo este comportamento em background consome um núcleo temporal com a seguinte anatomia prototipada:

```
public static String readGuess(Scanner scanner) {
    waitingForInput = true;

    Thread timer = new Thread(() -> {
        int time = 30; // 30 segundos (Tempo Limite)
        while (waitingForInput && time > 0) {
            try {
                Thread.sleep(1000);
                time--;
                if (waitingForInput) {
                    String timeStr = String.format("%02d", time);
                    // \033[s guarda cursor, \033[A navega p/ Norte e
                        reescreve
                    System.out.print("\033[s");
                    System.out.print("\033[" + linesToMoveUp + "A");
                    System.out.print("\r   RONDA " + currentRound + "    |
                        Tempo: " + timeStr + "s \033[K");
                    System.out.print("\033[u"); // \033[u liberta cursor para
                         o Sul
                    System.out.flush();
                }
            } catch (InterruptedException e) { break; }
        }
    });
    timer.setDaemon(true);
    timer.start();

    String result = null;
    if (scanner.hasNextLine()) {
        result = scanner.nextLine().trim();
    }

    waitingForInput = false;
    timer.interrupt();
    return result;
}
```

Listing 3.4: Rotina Assíncrona de Input com Relógio ANSI em Segundo Plano

Devido à melhoria visual proporcionada por este mecanismo durante o decorrer das rondas, a mesma ideia foi aplicada à contagem regressiva na fase inicial (**Lobby**). Assim, em vez de o terminal mostrar repetidamente mensagens como "Aguardar jogadores..."sempre que o tempo diminui ou entra um novo jogador, toda a informação é atualizada no mesmo local de forma assíncrona, reagindo a mensagens de rede do tipo `LOBBY_UPDATE`.

3.8.2 Renderização Gráfica do Boneco

A representação visual do jogo é feita através do clássico boneco da forca. Como as consolas de texto não permitem gráficos avançados, foi utilizada *Arte ASCII* para contornar essa limitação.

A lógica de desenho baseia-se no número de erros acumulados pelos jogadores (ou seja, $6 - \textit{tentativas restantes}$). Dependendo desse valor, partes do boneco vão sendo mostradas.

Para isso, são usadas condições (por exemplo, operadores ternários) que decidem quais caracteres devem aparecer, como a cabeça `'O'` ou os braços `'/'`. Caso o número de erros ainda não seja suficiente, esses elementos são substituídos por espaços em branco.

O seguinte bloco na `ConsoleUI` demonstra este funcionamento:

```java
public static void drawHangman(int attemptsLeft) {
    // Reduzimos as partes penduradas proporcionalmente a 6 fases
    int erroCount = 6 - (int) Math.ceil((double) attemptsLeft * 6 / Math.max(
        maxAttempts, 1));

    String head = erroCount >= 1 ? "O" : " ";
    String body = erroCount >= 2 ? "|" : " ";
    String lArm = erroCount >= 3 ? "/" : " ";
    String rArm = erroCount >= 4 ? "\\" : " ";
    String lLeg = erroCount >= 5 ? "/" : " ";
    String rLeg = erroCount >= 6 ? "\\" : " ";

    System.out.println("                  ");
    System.out.println("        " + head);
    System.out.println("       " + lArm + body + rArm);
    System.out.println("       " + lLeg + " " + rLeg);
    System.out.println("     ");
    System.out.println("                ");
}
```

Listing 3.5: Rotina dinâmica de Renderização do Boneco em ASCII

3.8.3 Apresentação de Letras Tentadas Anteriormente

A gestão do rol de perspetivas utilizadas (o inventário de letras submetidas falhadas ou já preenchidas) reveste-se de grande utilidade para prevenir castigos acidentais na alocação de letras erróneas ativas, garantindo visibilidade total na área de jogo.

Sob a ótica operacional distribuída da arquitetura implementada, os Clientes retêm o estado de forma leviana (*Stateless*). Todas as lógicas analíticas sobre as letras são processadas em *Cache* pelo sistema de controlo primário associado ao objeto `GameEngine` situado no Servidor Lógico e, em última instância, unificadas num vetor textual partilhado em rede pelo protocolo nativo `ROUND` e `STATE`. Assim que o cliente processa e captura a carga iterativa empacotada no tráfego, o mesmo apenas projeta o resultado puramente cru, injetado submetido de inspeção pela camada visual, indicando primeiro a validade material na *String* antes de desenhar a linha final com o respetivo limitador gráfico.

No que toca também ao temporizador, a integração da matriz ANSI tem em conta à priori a dimensão final se este bloco de letreiros visuais for impresso:

```java
// Se o pacote de strings enviado pelo servidor n o estiver em branco
if (!usedLetters.isBlank()) {
    System.out.println("  Letras usadas: " + usedLetters);
    linesToMoveUp = 12; // Ajuste preventivo para a navega  o ANSI
}
System.out.println(SEPARATOR);
System.out.print("  A tua jogada (letra ou palavra completa): ");
```

Listing 3.6: Sincronização passiva do Cliente com as Letras Usadas

Capítulo 4

Testes e Validação do Sistema

Para garantir a robustez da aplicação, foram realizados testes de stress e de comportamento em condições adversas, focando-se na experiência do utilizador e na estabilidade do servidor.

4.1 Gestão de Lotação (Servidor Cheio)

Um dos requisitos críticos é o controlo de acesso. O servidor limita o número máximo de jogadores a 4 ou impede a entrada após o início da partida. Quando um quinto jogador tenta conectar-se, o sistema envia a mensagem de protocolo `FULL` e encerra a conexão do lado do cliente de forma graciosa.

Como demonstrado na Figura 4.1, o cliente apresenta uma mensagem informativa, garantindo que o utilizador compreende a razão da recusa.

Figura 4.1: Interface do Cliente ao tentar entrar num lobby já completo ou em jogo.

4.2 Resiliência a Falhas de Rede (Desconexão)

O sistema foi testado quanto à sua capacidade de detetar quedas de ligação. Se um jogador fechar o terminal ou perder a conexão à rede, o servidor deteta a interrupção da *stream* de dados.

Como ilustrado na Figura 4.2, o sistema notifica os restantes utilizadores (através do comando `PLAYER_LEFT`) e, dependendo do estado, encerra a partida para evitar que o motor de jogo fique bloqueado a aguardar um *input* que nunca chegará.

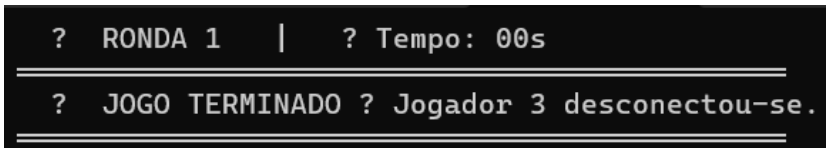

Figura 4.2: Notificação visual de desconexão inesperada de um dos participantes.

Capítulo 5

Conclusões e Trabalho Futuro

5.1 Conclusão

O desenvolvimento deste projeto permitiu consolidar os conhecimentos sobre comunicação em rede utilizando Sockets TCP e a gestão de concorrência em Java. A arquitetura cliente-servidor implementada mostrou-se robusta para o número de jogadores definido, garantindo que o protocolo de texto é seguido de forma rigorosa e síncrona. A separação de responsabilidades entre o motor de jogo e a gestão de comunicações facilitou a manutenção e o teste do sistema.

5.2 Trabalho Futuro

Para versões futuras deste sistema, poderiam ser implementadas as seguintes melhorias:

- **Interface Gráfica (GUI):** Substituir a consola por uma interface JavaFX para uma melhor experiência de utilizador.
- **Persistência de Dados:** Implementar uma base de dados para guardar um histórico de vitórias e recordes dos jogadores.

Bibliografia

[1] Daniel J. Berg and J. Steven Fritzinger. *Advanced Techniques for Java Developers*. John Wiley & Sons, 1999.

[2] James Gosling, Bill Joy, Guy Steele, and Gilad Bracha. *The Java Language Specification*. Addison-Wesley, 2000.

[3] Jeff Magee and Jeff Kramer. *Concurrency: State Models and Java Programs*. John Wiley & Sons, 1999.

[4] Oracle. Java platform, standard edition 17 api specification, 2021.

[5] Jon Postel. Transmission control protocol, 1981.

[6] Tiago Miguel Carrola Simões. Notas de aula e materiais de apoio de sistemas distribuídos. Universidade da Beira Interior (UBI), 2026. Acedido via plataforma EduMark e Moodle.